Sanvio Factory: onboarding cards

github.com/SanvioLabs/sanvio-factory

Six one-page cards, one for each session of getting a team onto the factory. Each builds on the one before, from the first commands to a loop that runs without anyone driving it. Each ends with what to do before the next card.

1 · Getting started	How to work with it, the setup that has to be right, and the commands you'll use every day
2 · Knowledge and skills	Connect one source of your own knowledge, write your first skill, and vet someone else's
3 · First motion	One real task from ticket to a pull request a person merged, through the loop
4 · Gates and guards	Turn the agent's real mistakes into rules and checks that block
5 · Specs	Move up from tasks to features: write, challenge, plan, build and review a spec
6 · Team and your loop	Bring the team in, keep a sprint rhythm, and let your loop run on its own

> Skills named `/stelliad:<name>` come from the Stelliad skills library, which the factory switches on: **github.com/Stelliad/stelliad-skills**. The rest are in this repo's `skills/`.

Card 1 · Getting started

Where everyone starts: how to work with it, the setup that has to be right, and the handful of commands you'll use every day. The cards after this one build on it.

> **The model is the engine.** Claude does the reasoning, the actions and the context handling.
>
> **The harness** is what one session works inside: the rules it reads, the skills it follows, the checks that stop it.
>
> **The factory** is everything between sessions: the queue of work, the loop that works it, and the review between a pull request and a merge. You decide what merges.

SETUP, ONCE PER CLONE

`git clone … my-factory`
Your own copy, on your own account

`git config core.hooksPath .githooks`
Skip it and every commit check silently does nothing

`python3 scripts/check_setup.py`
PASS, WARN or FAIL for each layer

Start Claude Code from the repo root and trust the folder. Anywhere else, the rules, guards and skills don't load.

HOW TO WORK WITH IT

- **Ask for an outcome**, and say what done looks like.
- **One job per session.** Finished, or going in circles? `/clear` and start fresh.
- **When it gets something wrong, say "learn from that."** Every mistake is a rule missing from the harness.
- **What must survive goes in a file.** A correction said out loud is gone next session.
- **It proposes, you decide.** Nothing merges without a person, unless the owner turns on auto-merge.

WHAT YOU'LL SEE

- **The lens**, a line above the prompt naming the part of the harness that just acted.
- **GitHub status**, a line with running actions, open pull requests and issues.
- **The guard**, before every action it takes: credential files stay unread, and anything through a connector that isn't a plain read asks you first. A block is the harness working, not a bug.

SANVIO SKILLS FOR THIS CARD

`/orientation`	Sets you up one layer at a time and writes a short record of your company
`/standup`	Morning read of what's on you, with a suggested order for the day
`/learn`	Turns the mistake it just made into a written rule
`/whats-new`	What changed in the factory, and what to do to take it
`/stelliad:secret-scan`	Finds exposed credentials before the agent reads your repos

You can also just ask: a skill runs when the request matches it. Stelliad skills come with the factory and run as `/stelliad:<name>`.

THREE CLAUDE COMMANDS

`/clear`	Start clean. A fresh context beats a long, muddled one
`/goal`	Set a goal and let it run until it's met: "write tests for these bugs, make them pass, get the review to approve"
`/loop`	Repeat a task on an interval: review open pull requests every 30 minutes, written to a folder you read later

BEFORE CARD 2

1. Run `/standup` each morning.
2. Say "learn from that" at least once, and find the rule it wrote.
3. Run `/stelliad:secret-scan` on each repo you plan to point it at.
4. Write down one procedure you do every week. It becomes your first skill on card 2.

Card 2 · Your knowledge and skills

Before it builds anything, it should know how you work. Connect one source of your own knowledge, then teach it a procedure you already run.

> **Knowledge** is what it reads: your standards, runbooks and templates, from the place people already look things up.
>
> **A skill** is what it does: one procedure you run the same way every time, written down so the agent runs it the same way too.

CONNECT ONE SOURCE, THE SAFE WAY

- **One named space**: a single wiki space or folder, not the whole wiki.
- **Read-only, through your own connector**: the wiki or drive tool you already use.
- **One row in `company/DATA.md`**: where it lives, how it's reached, whether it's cleared. `company/` stays on your machine.
- **The test**: ask it something only your company would know. It names the page.

If it can't name the page, it guessed. Also ask one thing your docs don't cover: it should say it can't find it.

WHAT NEVER GOES IN

- Customer or end-user records
- Credentials, tokens and keys: refer to them by name, never paste them
- Anything regulated, until your contract says it can
- Correspondence with third parties

If a space mixes standards with records, pick a narrower one.

A SKILL HAS FOUR PARTS

1. A name
2. A description that says **when** to use it
3. Numbered steps
4. A defined output. If you can't say what it is, it isn't a skill yet

Start from `examples/proposal/skills/draft-proposal/`: copy it and rewrite it for a weekly job, such as a status update, a release note, a code review or a support reply.

ONE CLAUDE COMMAND

`/mcp`	See the connectors this session has, and add the one for your knowledge source

SANVIO SKILLS FOR THIS CARD

`/orientation`	Lists what's connected, says where to add what isn't, and drafts the `DATA.md` row. Never logs in for you
`/review-skill`	Vets someone else's skill in quarantine and adopts only what fills a gap
`/inbox`	What in your mail needs a reply from you, once `DATA.md` clears the mailbox. Reads only
`/stelliad:repo-audit`	Checks a repo against a security and quality baseline before agents work in it

BORROWED SKILLS GET REVIEWED FIRST

A skill is someone else's instructions running with your permissions. Say "review this skill: <link>" and it sorts everything into:

Covered	You already have this
Gap	Adopt it, in your own conventions
Conflict	It fights a rule you have. Leave it out

Check the licence (MIT or Apache: adapt it; CC BY-SA: keep the licence; none: ideas only), then say "make me a local version of that, in our conventions" and commit it. Adopt the gaps, never the whole file.

STELLIAD SKILLS

A public skills library that comes with the factory, pinned to a commit that has already been reviewed. It's the one source you don't vet first. Run them as `/stelliad:<name>`, and browse the catalogue at **github.com/Stelliad/stelliad-skills**. Moving the pin to a newer commit is a review: read the diff first.

Also worth a look, reviewed first: `skill-creator` (Anthropic) writes and tests your own skills; `find-skill` searches public catalogues.

BEFORE CARD 3

1. Connect one space. Get two answers that name the page, and one "I can't find it".
2. Turn the weekly procedure from card 1 into a skill, and commit it.
3. Put one outside skill through `/review-skill`.
4. Pick one small, real task for card 3: a bug, a doc fix or a small feature.

Card 3 · First motion

One small, real task, end to end: written as a ticket, built by the loop, merged by a person. Run it on a sandbox repo, never a production one.

> **The loop** takes a labelled issue, plans it, builds it, tests it and has a second agent review it, then opens a pull request.
>
> **It doesn't merge.** A person reads the pull request and decides. Only the owner can change that, with `auto_merge` in `loop.json`, and then only for work it graded, by trusted authors, with every check passed.

FROM TICKET TO MERGE

1. **Write it**: `/stelliad:create-ticket` turns a rough idea into an issue.
2. **Grade it**: `/stelliad:review-ticket` says READY, or exactly what's missing.
3. **Label it `loop-ready`**, or say "put #12 on the loop". The label is the gate.
4. **The loop works it**: plan, build, test, review.
5. **It opens a pull request.** You read it, then merge it or send it back.

NEEDS_REFINEMENT is a good result: fix what it names and grade it again. Label only issues you wrote or have read, because a ticket is an instruction to an agent.

WHAT THE LOOP DOES

Discover	Picks the labelled issue and writes `ticket.md`
Shape	A fresh agent writes `plan.md`
Build	Another fresh agent makes the change in its own worktree
Validate	The script runs your tests; a second agent reads the diff. FIX sends it back to Build
Scale	Opens the pull request; what went wrong goes to `.loop/lessons.md`

WATCHING IT

- `/factory-floor`: the queue, each issue's step and round, tokens and estimated cost.
- `tail -f .loop/loop.log`: one plain line per step.
- Each pull request records its own tokens and cost. `max_issue_tokens` in `loop.json` stops an issue at a budget.

SANVIO SKILLS FOR THIS CARD

`/stelliad:create-ticket`	A rough idea into a ticket nobody has to come back and ask about
`/stelliad:review-ticket`	Grades it READY, NEEDS_REFINEMENT or BLOCKED
`/loop-control`	"Is #12 ready?", "start the loop", "why is #12 blocked?", "stop the loop"
`/review-pr`	Reviews a pull request against your rules, findings ranked by cost. Never approves or merges
`/stelliad:review-intake`	Works through review comments: fixes what's right, pushes back with evidence on what isn't
`/stelliad:verify-done`	"Done" means the command that proves it ran and passed

READ IT LIKE A REVIEWER

- **The plan**: did it understand the ticket?
- **The diff**: is this the change you'd have made?
- **The tests**: do they prove the change, or only that the code runs?
- **The second agent's review**: what did it flag, and was it right?

Whatever you had to fix by hand is a rule or a gate that's missing. That's card 4.

PICK THE RIGHT FIRST TASK

- Small enough to finish in one loop run: a bug, a doc fix, a small feature.
- Done is something a test can check.
- Real work, but nothing anyone is waiting on today.

BEFORE CARD 4

1. Take one real ticket from idea to a pull request you merged.
2. Write down every fix you made by hand, and every place it got something wrong.
3. Run `/review-pr` on one pull request a person wrote, and compare notes.
4. Put a second ticket on the loop without help.

Card 4 · Gates and guards

Add checks where the agent actually went wrong, not where you guess it might. Your list of hand fixes from card 3 is the input.

> **A rule is context.** The agent reads it, and can still skip it.
>
> **A gate is enforcement.** A script that fails when work isn't ready, so "done" is something a script checks, not something the agent says.
>
> **A check that doesn't block is decoration.**

MISTAKE TO RULE

1. **Fix the output**, then don't stop there.
2. **Say "learn from that."** The rule goes into the file that governs it, dated, with the reason.
3. **Not sure why it went wrong?** `/retro` reads that session's own log and says why the agent didn't know.

A correction said out loud lasts one conversation. A rule in the file lasts every session after.

WHERE EACH FIX GOES

`AGENTS.md`, `steering/`	It didn't know how you work
A skill's steps	A procedure missed a step
A gate	It has to hold even when the agent decides otherwise
A review agent	It needs judgement, not a script

A check earns a gate when a script can decide it and missing it costs more than checking it.

WHERE A GATE RUNS

A skill's last step	The cheapest catch: the agent fixes it before you see the work. But an agent can skip a step
The git hook	Every commit, from any tool. Unless the hook is off, or someone uses `--no-verify`
CI	Every push. Once it's a required check on `main`, nobody can skip it

EXAMPLE TO COPY

`examples/proposal/gates/proposal_gate.py` blocks a commit when a rate isn't the rate card's, a total doesn't add up, or a placeholder is left in. One file, wired into the skill, the hook and CI.

SANVIO SKILLS FOR THIS CARD

`/learn`	Turns the mistake it just made into a written rule, in the right file
`/retro`	Why it went wrong, from the session log. Turn it on with `SANVIO_RETRO=on`
`/review-tests`	Breaks the code on purpose and shows which tests actually catch it
`/stelliad:test-first`	No code without a failing test first
`/stelliad:run-gates`	Turns your checkpoints into checks that block instead of advise
`/stelliad:coverage-gaps`	Which risky code no test touches
`/stelliad:review-principles`	Reviews code for DRY, KISS and single responsibility, each finding tied to a consequence

RULES OF THE ROAD

- **Never edit a gate to make work pass.** Fix the work.
- **A pass goes stale.** A check that passed yesterday says nothing about this morning's edit. That's why the hook runs on every commit.
- **Tests an agent wrote get tested too.** Passing isn't the same as protecting.

BEFORE CARD 5

1. Write one gate for a mistake from card 3, dated with the reason, and wire it into the hook.
2. Try to commit something that should fail, and watch it refuse.
3. Run `/review-tests` on tests the agent wrote.
4. Make CI a required check on `main`, so it blocks instead of reports.

Card 5 · Specs

Move up from tasks to features. Write down what has to be true before anyone builds it, then build it one tested task at a time.

> **A ticket** is one task the loop can finish in a single run.
>
> **A spec** is a feature: what has to be true, how anyone can tell, and what's still undecided.
>
> **Two ways to feed the factory:** a spec, or well-defined tickets. Both work.

THE SPEC LOOP

1. Write	`create-spec` drafts `SPEC.md` from what was actually said. What wasn't said stays an open question
2. Challenge	`stress-test-plan` questions it one round at a time until the weakest assumption is named
3. Plan	`plan-spec` writes `PLAN.md` and `TASKS.md`. Every task names its requirements and a check that really exists. It never touches code
4. Build	`implement-spec` works one task at a time, test first, and stops when the spec turns out to be wrong
5. Review	`review-spec` walks every requirement to the code and test that meet it

WHERE A SPEC STARTS

- **Your notes.** "Help me build a spec from these meeting notes."
- **Feature size, not four-hour tasks.** The agent does far more per task than a developer did.
- **Code with no spec?** `recover-spec` works backwards from what's built, and lists what nobody ever decided.

FROM SPEC TO THE LOOP

A task can go to the loop as a ticket with a `Spec: docs/specs/<name>` line. The ticket's shape is in `.github/ISSUE_TEMPLATE/loop-task.md`: objective, why, acceptance criteria, scope in and out, risk.

SANVIO SKILLS FOR THIS CARD

`/stelliad:create-spec`	An idea into a draft spec, open questions left open
`/stelliad:stress-test-plan`	Finds the weakest assumption before you build on it
`/stelliad:plan-spec`	A ready spec into a plan and a task list
`/stelliad:implement-spec`	The next unblocked task, test first and verified
`/stelliad:review-spec`	Independent final review before a spec is done
`/stelliad:recover-spec`	The spec an existing codebase never had

The whole spec loop is Stelliad, so it comes with the factory.

ONE CLAUDE MOVE

`Shift+Tab`	Plan mode: it reads and plans, and changes nothing. Use it to think a feature through before a spec exists

RULES OF THE ROAD

- **An open question is better than a guess.** A spec that guesses gets built on the guess.
- **When the spec is wrong, fix the spec first**, then the code.
- **The spec lives in the repo**, next to the code it governs.
- **A plan's check has to run.** A task whose check doesn't exist isn't planned yet.

BEFORE CARD 6

1. Draft one spec from real notes, for a real feature.
2. Stress-test it, and answer what it asks.
3. Plan it, then build the first two tasks, here or on the loop.
4. Name the first person on your team who joins next.

Card 6 · Team and your loop

One person first, then the team. Then a loop that works through tickets and specs without anyone driving it, with a person at the end.

> **The owner owns the direction. The team pushes the edges.** People experiment freely inside the rails that matter, and bring back what works.
>
> **Every sprint, the experiments come back**, and the owner decides what goes into the factory everyone shares.

BRINGING THE TEAM IN

1. **Before:** everyone has cloned it and the setup check passes.
2. **Open with why:** what the owner has stopped doing by hand, with real examples.
3. **One live motion:** the owner takes a small real task from spec to pull request while the team watches.
4. **Everyone tries one thing:** a skill, a review, a ticket.
5. **Close with the rules:** what's shared, what's yours to change, and how experiments come back.

THE SPRINT RHYTHM

- **Usage is visible.** Each person's sessions log which skills ran and where the guard stepped in: names and counts, never content, on their own machine. `SANVIO_USAGE_LOG=off` turns it off.
- **At sprint end**, `/factory-trends` turns that into a short report with proposals, shared as a pull request.
- **The owner decides** what goes in, and what gets dropped.

YOUR LOOP, ON ITS OWN

`loop.py run`	Takes each `loop-ready` issue as it's labelled, until stopped. Bound it with `--for 8h` or `--hours "Mon-Fri 09:00-17:00"`
`loop.py review`	Reviews each pull request that asks for you, once per commit
`max_workers`	Several issues at once, in `loop.json`. Usage grows with each worker
`loop.py stop`	Stops after the current step

SANVIO SKILLS FOR THIS CARD

`/factory-trends`	A sprint's usage as trends, with one recommended change
`/pulse`	The team's daily one-question check-in, with yesterday's answers as counts
`/work-report`	One page of what merged, who approved it, and what the loop built
`/dispatch`	Hand an issue to another team's factory, and review what comes back
`/factory-updates`	What the public factory added since your copy, and what's worth taking
`/factory-dashboard`	A pane, not a skill: what's waiting on you, the loop's spend and rework, token use. On by default
`/factory-eval`	Whether a change made the factory better: runs a suite, scores it, compares two runs

The review agents (`reviewer`, `security-reviewer`, `cto`, `gc`, `ceo`, `cmo`) each read the work from one angle, and never edit it.

CLAUDE COMMANDS

`/model` · `/advisor`	Pair a cheaper model that builds with a stronger one that advises. Move up only when the work gets hard

SIGNS IT'S WORKING

- Pull request review turnaround drops.
- Tests the agent writes catch real breaks.
- You mostly just say "continue".
- Token use rises with output, not without it.
- Skills used, not just installed.

YOU'RE RUNNING IT WHEN

1. The owner runs the sprint review and decides what goes in.
2. Work flows through your own loop, with a person at the end.
3. The team brings proposals each sprint without being asked.
4. Your rules and gates come from your own mistakes, not just the factory's.